

EPISODE 5

Production Systems, Infrastructure & Deployment

From Localhost to the Cloud

DevOps Foundations

Docker & Containers

AWS EC2 & Deployment

Security Groups & IAM

Load Balancers

CloudFront CDN

CI/CD with GitHub Actions

Scaling & Reliability

TABLE OF CONTENTS

1 DevOps Foundations

- CI/CD concepts
- Dev vs Staging vs Production
- Linux & SSH basics

2 Docker & Containerization

- Images vs Containers
- Writing Dockerfiles
- Docker Compose

3 Cloud Computing Basics

- Cloud providers
- Scalability & availability

4 AWS EC2

- Launching instances
- Connecting via SSH
- Installing Node & Docker

5 Security Groups

- Inbound/outbound rules
- Least privilege
- Best practices

6 Load Balancers

- ALB setup
- Health checks
- Target groups

7 AWS CloudFront

- CDN setup
- Edge caching
- SSL with ACM

8 CI/CD with GitHub Actions

- Workflow files
- Test & build
- SSH deploy

9 Scalability & Reliability

- Vertical vs horizontal
- Stateless design
- Auto Scaling

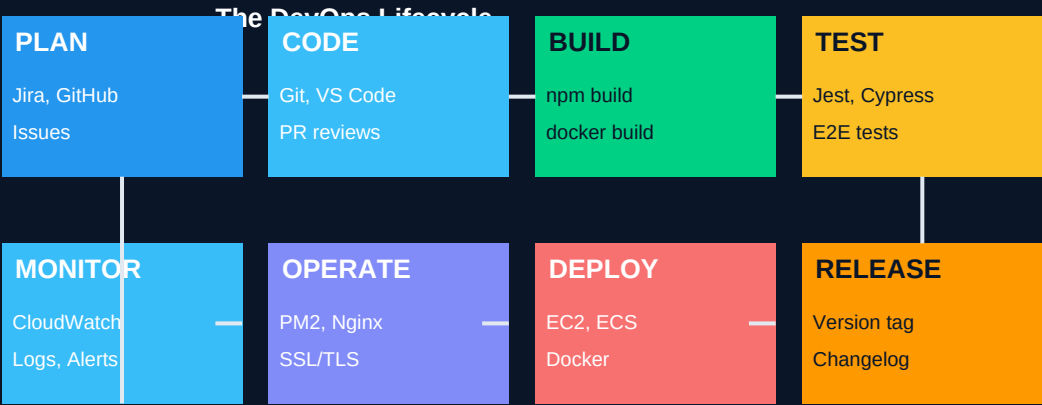
10 Monitoring & Observability

- Logs, metrics, traces
- CloudWatch
- Alerting

CHAPTER 1 — DEVOPS FOUNDATIONS

What is DevOps?

DevOps is the practice of combining software Development and IT Operations. It breaks the wall between 'the people who write code' and 'the people who run it.' The goal: ship software faster, more reliably, and with fewer production surprises.



CI = Code→Build→Test | CD = Release→Deploy

DevOps lifecycle: code → build → test → release → deploy → operate → monitor → repeat

Aspect	Development	Production
Environment	localhost:3000	203.0.113.42 on EC2
Data	Fake/seed data	Real user data
Errors	Visible stack traces	Hidden from users
Logging	console.log	Structured JSON to CloudWatch
Config	Hardcoded .env	Secrets Manager / SSM
Restarts	Manual	PM2 / ECS auto-restart
Secrets	In .env file	Env vars injected at runtime

Essential Linux Commands

```
# Navigation & files
ls -la          # list all files with permissions
cd /var/www/app # change directory
mkdir -p src/config # create nested directories
cp -r src/ backup/ # copy directory recursively
rm -rf node_modules # remove directory (careful!)

# Process management
ps aux | grep node # find running Node processes
kill -9 <PID>      # force kill a process
top               # live process monitor (q to quit)
htop              # better top (install separately)

# File editing
nano .env         # simple editor (Ctrl+X to exit)
vim server.js     # advanced editor (:wq to save+quit)

# Networking
curl http://localhost:3000/health # test API locally
netstat -tulpn | grep :3000      # what's on port 3000?
ufw allow 3000/tcp                # open firewall port

# SSH into remote server
ssh -i ~/.ssh/my-key.pem ubuntu@203.0.113.42

# File transfer
scp -i my-key.pem app.tar.gz ubuntu@server:/home/ubuntu/
```

I

CHAPTER 2 — DOCKER & CONTAINERIZATION

Why Docker?

'It works on my machine' is a famous developer phrase. Docker eliminates this by packaging your app AND its environment into a container — the same image runs identically on your laptop, CI server, and AWS. No more 'works locally, broken in prod'.

Aspect	Traditional VMs	Docker Containers
Startup time	Minutes	Seconds
Size	GBs (full OS)	MBs (shared kernel)
Isolation	Full OS isolation	Process isolation
Portability	Limited	Run anywhere Docker runs
Resource use	Heavy (dedicated RAM/CPU)	Lightweight (shared kernel)
Use case	Legacy apps, strong isolation	Microservices, CI/CD

Docker Image Layers (Union Filesystem)

Base OS layer

FROM node:18-alpine (read-only)

Cached!

Dependencies

COPY package*.json ./ && RUN npm ci (read-only)

Cached!

Application code

COPY src/ ./src/ (read-only)

Rebuilds

Config / env

ENV NODE_ENV=production (read-only)

Rebuilds

Container layer

Read-write – your app runs here

Tip: Put COPY package.json BEFORE COPY src/ — cache npm install!

Docker image layers are cached — order matters! Stable layers first for fast rebuilds

```
# Optimized Dockerfile for Node.js
FROM node:18-alpine

WORKDIR /app

# Copy package files FIRST (cached layer – only invalidates when deps change)
COPY package*.json ./
RUN npm ci --only=production

# Copy source AFTER deps (only this layer rebuilds when code changes)
COPY src/ ./src/

# Security: don't run as root
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
USER appuser

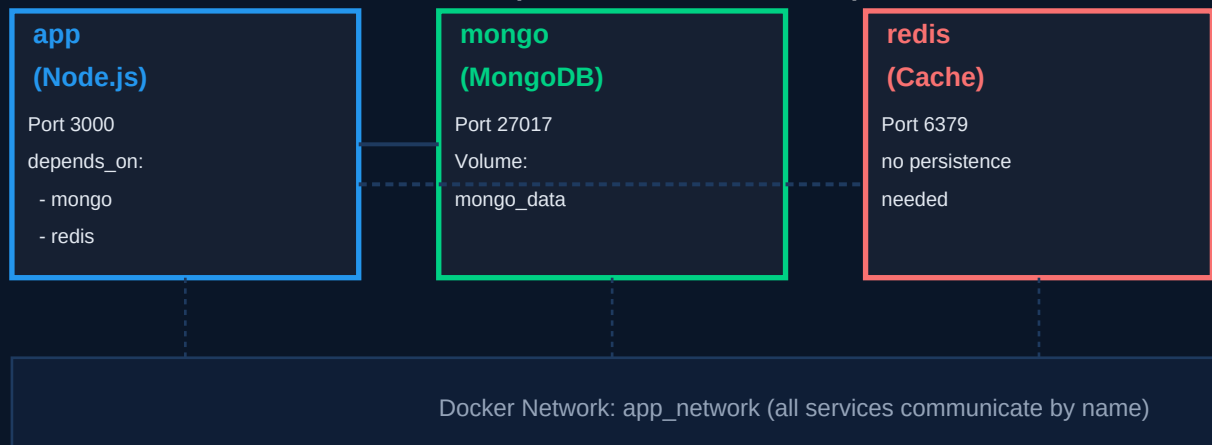
EXPOSE 3000
ENV NODE_ENV=production

CMD ["node", "src/index.js"]

# .dockerignore – exclude from build context
# node_modules
# .env
# .git
# *.test.js

# Build & run
docker build -t my-api:1.0.0 .
docker run -p 3000:3000 --env-file .env my-api:1.0.0
docker ps                # list running containers
docker logs <container-id> -f  # follow logs
```

Docker Compose — Multi-Service Setup



Docker Compose: define and run multi-container apps with one command

```
# docker-compose.yml
version: '3.8'
services:
  app:
    build: .
    ports: ['3000:3000']
    environment:
      - NODE_ENV=production
      - MONGO_URI=mongodb://mongo:27017/mydb
      - REDIS_URL=redis://redis:6379
    depends_on: [mongo, redis]
    restart: unless-stopped

  mongo:
    image: mongo:7
    volumes: [mongo_data:/data/db]
    restart: unless-stopped

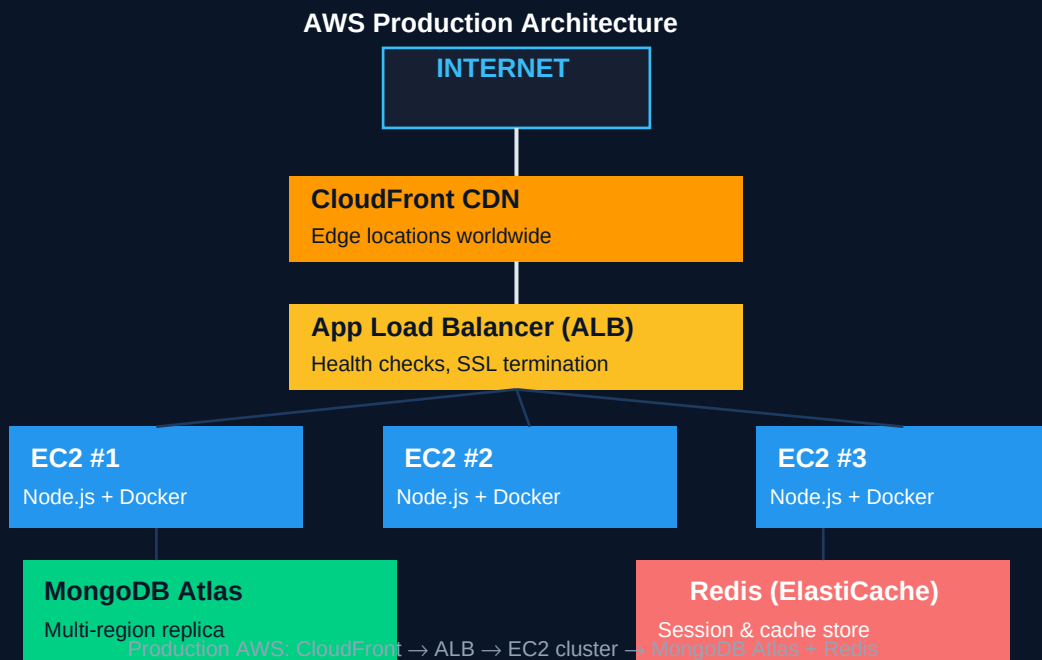
  redis:
    image: redis:7-alpine
    restart: unless-stopped

volumes:
  mongo_data:

# Commands
docker compose up -d          # start all services (detached)
docker compose down           # stop all
docker compose logs app -f    # follow app logs
docker compose exec app sh     # shell into app container
```

CHAPTERS 3 & 4 — CLOUD COMPUTING & AWS EC2

AWS Production Architecture




```
# Step-by-step: Launch & configure EC2

# 1. Launch EC2 (Ubuntu 22.04 LTS, t2.micro for dev / t3.medium for prod)
# Via AWS Console: EC2 → Launch Instance → Ubuntu → t2.micro → Key pair

# 2. Connect via SSH
chmod 400 my-key.pem    # secure the key file
ssh -i my-key.pem ubuntu@<EC2-PUBLIC-IP>

# 3. Update system & install Node.js
sudo apt update && sudo apt upgrade -y
curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash -
sudo apt install -y nodejs git

# 4. Install Docker
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker ubuntu    # allow ubuntu user to run docker
newgrp docker                      # apply without re-login

# 5. Install PM2 (production process manager)
sudo npm install -g pm2

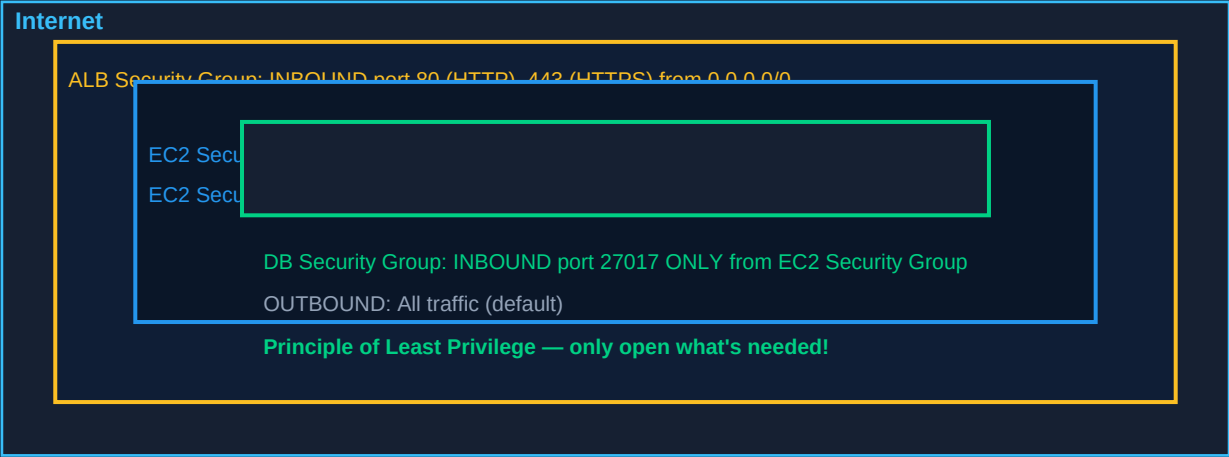
# 6. Deploy your app
git clone https://github.com/yourname/my-api.git
cd my-api
cp .env.example .env && nano .env    # set production env vars
npm ci --only=production
pm2 start src/index.js --name 'api'
pm2 save && pm2 startup                # auto-start on reboot

# 7. OR use Docker
docker compose up -d                # start all services
```

CHAPTERS 5 & 6 — SECURITY GROUPS & LOAD BALANCERS

Network Security

AWS Security Groups — Firewall Rules

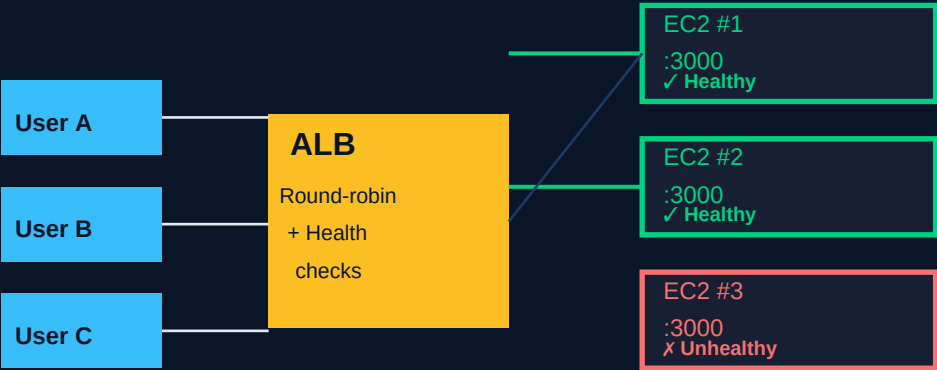


Security Groups are stateful firewalls — apply least-privilege at every layer

Port	Protocol	Source	Purpose
80	HTTP	0.0.0.0/0 (anyone)	HTTP traffic → redirect to 443
443	HTTPS	0.0.0.0/0 (anyone)	Production HTTPS traffic
22	SSH	YOUR IP only	Admin access — never 0.0.0.0/0!
3000	HTTP	ALB Security Group only	App server port
27017	TCP	EC2 Security Group only	MongoDB
6379	TCP	EC2 Security Group only	Redis

Application Load Balancer

Load Balancer — Traffic Distribution

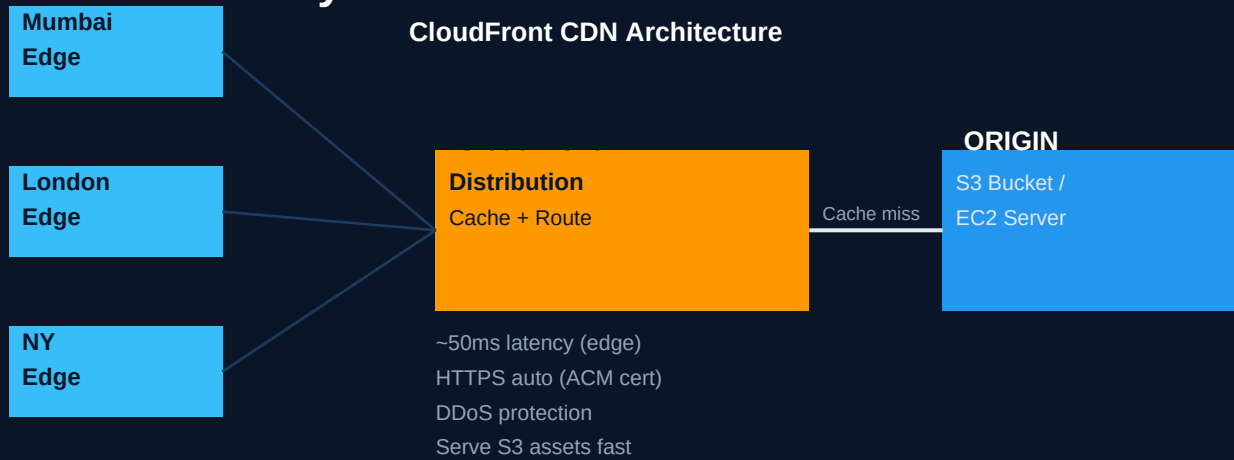


Unhealthy → ALB stops routing to it automatically

ALB distributes traffic across healthy EC2 instances — auto-removes unhealthy ones

CHAPTER 7 — AWS CLOUDFRONT CDN

Content Delivery Network



CloudFront: serve cached content from 400+ edge locations — milliseconds anywhere

```

# CloudFront setup for Next.js / static assets

# 1. Create S3 bucket for static assets
aws s3 mb s3://my-app-assets
aws s3 sync ./public s3://my-app-assets --delete

# 2. Create CloudFront distribution (via console or CLI)
# Origin: S3 bucket / EC2 ALB DNS
# Behaviors: /api/* → EC2 (no cache)
#           /* → S3 (cache images/CSS/JS)

# 3. SSL Certificate via ACM (free!)
# Request cert for myapp.com + www.myapp.com
# Validate via DNS (add CNAME to Route53)

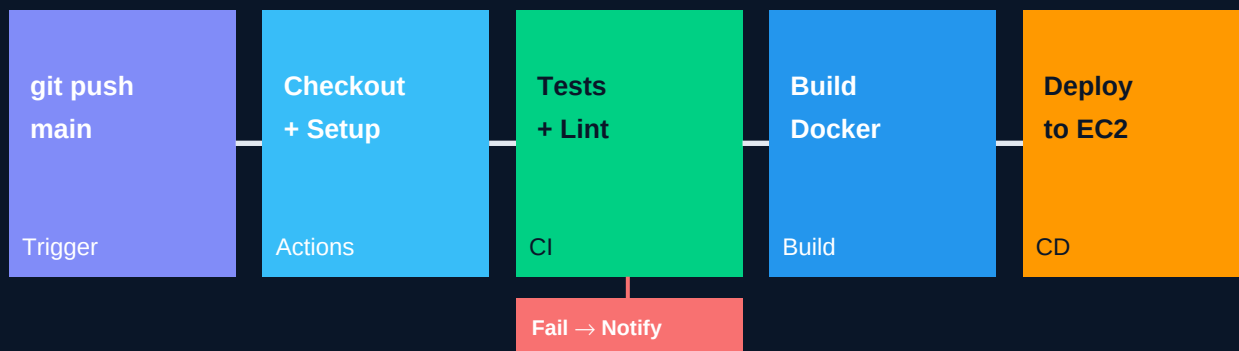
# 4. Cache headers in Express (for CDN to cache properly)
app.use('/static', express.static('public', {
  maxAge: '1y',          // browser caches for 1 year
  etag: true,
  lastModified: true
}));

// API responses should NOT be cached
app.use('/api', (req, res, next) => {
  res.setHeader('Cache-Control', 'no-store');
  next();
});
  
```

CHAPTER 8 — CI/CD WITH GITHUB ACTIONS

Automated Deploy Pipeline

GitHub Actions CI/CD Pipeline



On push to main: run all tests → build image → push to ECR → SSH deploy to EC2

CI/CD: every git push triggers test → build → deploy automatically

```
# .github/workflows/deploy.yml
name: Test, Build & Deploy

on:
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '18' }
      - run: npm ci
      - run: npm test
      - run: npm run lint

  build-and-deploy:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      # Build & push Docker image to ECR
      - name: Configure AWS credentials
        uses: aws-actions/configure-aws-credentials@v4
        with:
          aws-access-key-id: ${ secrets.AWS_ACCESS_KEY_ID }
          aws-secret-access-key: ${ secrets.AWS_SECRET_ACCESS_KEY }
          aws-region: ap-south-1

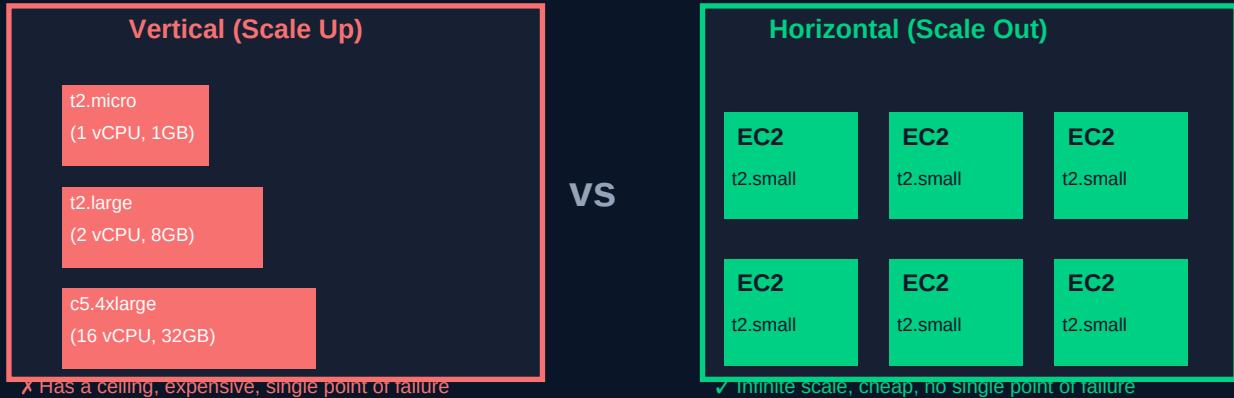
      - name: Build & push to ECR
        run: |
          aws ecr get-login-password | docker login --username AWS ...
          docker build -t my-api:${ github.sha } .
          docker push $ECR_REGISTRY/my-api:${ github.sha }

      # SSH into EC2 and deploy
      - name: Deploy to EC2
        uses: appleboy/ssh-action@v1
        with:
          host: ${ secrets.EC2_HOST }
          username: ubuntu
          key: ${ secrets.EC2_SSH_KEY }
          script: |
            cd /home/ubuntu/my-api
            git pull origin main
            docker compose pull
            docker compose up -d --no-deps app
            docker system prune -f
```

CHAPTERS 9 & 10 — SCALING & MONITORING

Scalability Strategies

Vertical vs Horizontal Scaling



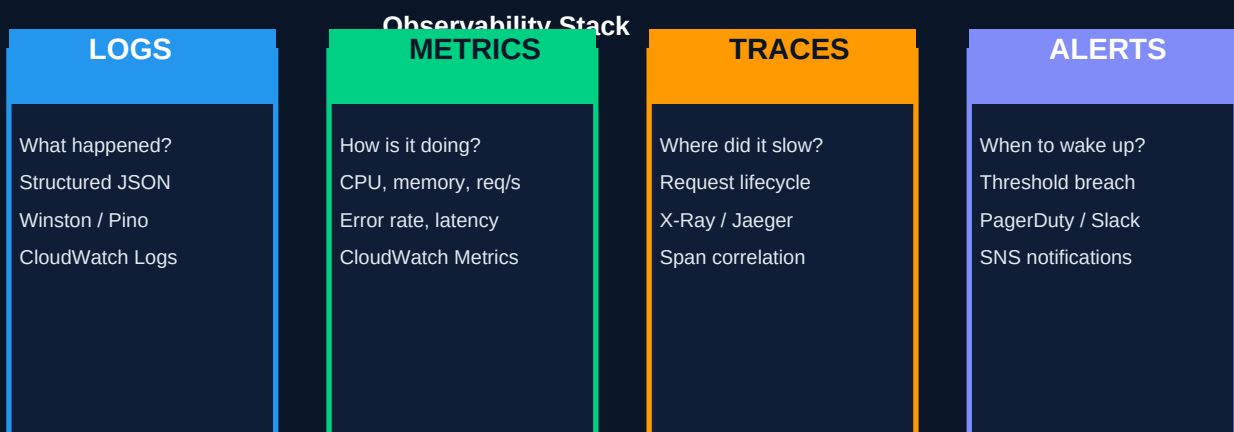
Horizontal scaling is the cloud-native approach — stateless services behind a load balancer

Stateless Design — Required for Horizontal Scaling

Stateless: each request contains ALL the info needed — no server-side session.

- ✓ Store sessions in Redis (shared across all instances).
- ✓ Store uploads in S3 (not local disk — each EC2 has its own disk).
- ✓ Use JWT tokens (self-contained, no server-side session storage).
- ✓ Store config in env vars, not in-memory (different instances, different memory).

Rule: if you killed an EC2 instance randomly, should the app still work? YES!



Observability = Logs + Metrics + Traces — you need all three in production

```
// Structured logging with Winston
const winston = require('winston');

const logger = winston.createLogger({
  level: process.env.LOG_LEVEL || 'info',
  format: winston.format.combine(
    winston.format.timestamp(),
    winston.format.json() // structured JSON for CloudWatch
  ),
  transports: [
    new winston.transports.Console(),
    new winston.transports.File({ filename: 'error.log', level: 'error' })
  ],
});

// Structured log entry (searchable in CloudWatch)
logger.info('User login', {
  userId: user.id,
  email: user.email,
  ip: req.ip,
  userAgent: req.headers['user-agent'],
  duration_ms: Date.now() - startTime
});

// Request logging middleware
app.use((req, res, next) => {
  const start = Date.now();
  res.on('finish', () => {
    logger.info('HTTP Request', {
      method: req.method, url: req.url,
      status: res.statusCode,
      duration_ms: Date.now() - start
    });
  });
  next();
});
```

EPISODE 5 COMPLETE — WHAT YOU'VE LEARNED

✓ DevOps Lifecycle	CI/CD concepts, environments, Linux commands, SSH
✓ Docker	Images vs containers, Dockerfile best practices, docker-compose
✓ AWS EC2	Launch, SSH access, Node.js install, PM2, Docker on EC2
✓ Security Groups	Least privilege, port rules, layered network security
✓ Load Balancers	ALB setup, health checks, round-robin distribution
✓ CloudFront CDN	Edge caching, SSL via ACM, cache-control headers
✓ GitHub Actions	Workflow YAML, test/build/deploy pipeline, secrets
✓ Horizontal Scaling	Stateless design, Redis sessions, S3 for uploads

✓ Monitoring

Structured logging, Winston, CloudWatch, alerts

✓ Production Checklist

HTTPS, env vars, error boundaries, graceful shutdown

What's Next — Episodes 6, 7, 8 & 9

Episode 6: Microservices, Scaling & Web3 Basics

Episode 7: Data Structures & Algorithms (DSA) with JavaScript

Episode 8: Aptitude & Reasoning (Interview Prep)

Episode 9: System Design — Low Level + High Level Design